

PAW

Power Optimization Strategy

Detailed, Point-Wise Design Rationale

1000 mAh & 2000 mAh Li-Ion Scenarios — Current Draw Still an Engineering Estimate

Version 1.0

September 2026

1. Overview

The PAW collar's power strategy rests on keeping the MCU and its peripherals idle by default and only active for the minimum time a task requires. This document breaks that strategy into seven individual design decisions, ties each one to the specific firmware construct that implements it, and then addresses battery life directly. Section 6 estimates runtime for a 1000 mAh Li-Ion cell and Section 7 repeats the same calculation for a 2000 mAh alternative, both using clearly labeled datasheet-typical assumptions in place of measured current draw, since that and real-world duty cycle are still not measured on physical hardware. Sections 4 and 5 explain that gap and what closing it would require.

2. Power Optimization Techniques

2.1 Event-Driven Architecture — No Polling Loop

The firmware never busy-waits on a sensor or timer. The MCU's main loop does exactly one thing:

- `loop()` body is a single call to `api.system.sleep.all()` — there is no polling of GPIO state, sensor registers, or timers in the main loop.
- All work is dispatched from two sources only: a hardware interrupt (accelerometer) or a software timer callback (periodic fix) — both defined once in `setup()` and never re-armed manually.
- This matters because a polling loop keeps the CPU clocked and active continuously, which is one of the largest avoidable current draws on a battery-powered MCU; an interrupt/timer-driven design lets the MCU's own low-power sleep state handle idle time instead of firmware logic.

2.2 Accelerometer Interrupt Instead of Continuous MCU Polling

Motion detection is delegated entirely to the LIS2DW12, which runs its own low-power wake-up detection independently of the MCU:

- `LIS2DW12_CTRL1 = 0x10` configures the accelerometer's own low-power output data rate (12.5 Hz) — it evaluates the motion threshold (`LIS2DW12_WAKE_UP_THS`) on-chip, without any help from the MCU.
- `attachInterrupt(ACC_INT_PIN, accelerometerISR, RISING)` wires the accelerometer's INT1 pin directly to a hardware interrupt — the MCU only wakes when motion has already been detected, not to check whether it occurred.
- The ISR itself is minimal by design: it sets a flag and starts a one-shot software timer (`api.system.timer.start(RAK_TIMER_1, ...)`), deferring all I2C, GPS, and radio work to `motionHandler()` running outside interrupt context. Keeping the ISR short avoids blocking other interrupts and avoids doing slow bus transactions (I2C/UART) inside interrupt context.

2.3 GPS Activated Only When a Fix Is Actually Required

The GPS receiver is not queried opportunistically or on a fast background schedule — only when a location report is about to be sent:

- `hal_gps_acquire_fix()` is called exclusively from `motionHandler()` and `periodicHandler()` — i.e. only at the moment a transmission is being prepared, never speculatively.
- Acquisition is bounded by `GPS_FIX_TIMEOUT_MS` (60,000 ms) — if a fix cannot be obtained within 60 seconds, the attempt is abandoned rather than left running indefinitely, capping the worst-case GPS-on duration per event.
- A build-time mock mode (`HW_GPS MOCK`) exists purely for bench development without GPS hardware attached; it has no bearing on the production power design, where real GPS acquisition follows the same bounded on-demand pattern.

2.4 LoRa Radio Enabled Only During Transmission

The radio is not left in a listening or idle-powered state between events:

- `hal_lora_send()` / `api.lora.psend()` is invoked once per completed acquire-and-report cycle — a single bounded transmission, not a maintained radio session.
- TX power is fixed at 14 dBm (`LORA_TX_POWER`) — a deliberate mid-range setting rather than the radio's maximum; higher power extends range at direct energy cost per packet, so this value should be revisited once real-world link-margin needs are known.
- The payload is a fixed, compact 18 bytes (`PACKET_LEN`) — at SF9/CR 4/5, on-air time scales with payload size, so keeping the packet small directly shortens radio-on time per transmission.

2.5 MCU Returns to Low-Power Mode After Every Transmission

Both event handlers are designed to finish and hand control back to sleep, not to linger:

- `motionHandler()` and `periodicHandler()` are one-shot service routines — once `transmitLocation()` returns, execution falls back through `loop()` into `api.system.sleep.all()` with no intermediate active-wait step.
- `api.system.lpm.set(1)` and `api.system.lpm.lvl.set(2)` (the RAK3172's deepest supported low-power level) are configured once in `setup()`, so the MCU's baseline idle current is minimized by default rather than needing to be re-entered manually after each event.

Implementation note: The current firmware does not send an explicit power-down/standby command to the GPS module between fixes — UART inactivity alone does not guarantee a GNSS chip drops to a low-power state on all modules. This should be verified against the MAX-M10S's power management interface before production, as it directly affects the sleep-current figure used in any battery-life calculation.

2.6 Motion Transmission Cooldown (Debounce)

Motion detection and location reporting are deliberately decoupled so that continuous activity cannot flood the radio:

- `MOTION_COOLDOWN_MS` (120,000 ms / 2 minutes) suppresses a new motion-triggered GPS+LoRa cycle if one already ran within the cooldown window, even if the accelerometer keeps firing interrupts.
- Without this, a continuously active animal could cross the motion threshold many times per minute, each one triggering a full GPS acquisition and LoRa transmission — by far the two most energy-expensive operations in the whole cycle.
- The cooldown effectively bounds the worst-case duty cycle from motion alone, independent of how active the animal actually is, which is the single biggest lever this firmware has over real-world battery drain.

2.7 Guaranteed Maximum 30-Minute Periodic Update

A fixed floor on update frequency exists independently of motion activity:

- `MAX_FIX_INTERVAL_MS` (1,800,000 ms / 30 minutes) drives a periodic timer that runs regardless of whether any motion has occurred — the collar is never silent for hours if the animal is resting.
- This is a deliberate trade-off: the 30-minute floor adds a small, fixed, predictable amount of active-current use per day, in exchange for a bounded worst-case reporting gap — unlike the motion path, this component of power draw is not data-dependent and can be estimated precisely once per-event current is measured.
- A secondary, currently unverified benefit: regular fixes may keep the GPS receiver's ephemeris data warm, which typically shortens time-to-fix (and therefore GPS-on current-time) on the next acquisition — this depends on the MAX-M10S's specific warm-start behavior and has not been measured on this design.

3. Implementation Cross-Reference

Each strategy point above maps to a specific, verifiable construct in `collar_firmware.cpp`:

Strategy Point	Firmware Construct	Location
Event-driven architecture	<code>loop() → api.system.sleep.all()</code>	Main loop
Accelerometer interrupt	<code>attachInterrupt(ACC_INT_PIN, accelerometerISR, RISING)</code>	<code>hal_accel_init()</code>
On-demand GPS	<code>hal_gps_acquire_fix(), GPS_FIX_TIMEOUT_MS</code>	<code>motionHandler(), periodicHandler()</code>
Bounded LoRa TX	<code>hal_lora_send(), LORA_TX_POWER, PACKET_LEN</code>	<code>transmitLocation()</code>
Return to sleep	<code>api.system.lpm.set(1), api.system.lpm.lvl.set(2)</code>	<code>setup()</code>
Motion cooldown	<code>MOTION_COOLDOWN_MS</code>	<code>motionHandler()</code>
30-minute floor	<code>MAX_FIX_INTERVAL_MS, RAK_TIMER_0</code>	<code>setup(), periodicHandler()</code>

4. Battery Life: Why a Single Figure Is Still Not Stated

A defensible battery-life figure depends on three inputs, combined over time. One of the three is now known; the other two have not been measured on physical hardware for this design:

- Battery capacity (mAh) — confirmed as 1000 mAh Li-Ion (see Section 6).
- Measured current draw per state — sleep current, GPS-acquisition current, and LoRa TX current are, at best, typical datasheet figures until measured on the assembled board; actual draw is affected by board revision, antenna matching, voltage regulator efficiency, and temperature, none of which a datasheet number captures.
- Real-world duty cycle — the 30-minute periodic floor is fixed and predictable, but the motion-triggered rate depends entirely on how active a given animal actually is, which varies by animal and environment and cannot be assumed.

Position taken in this document: With capacity now known but current draw and duty cycle still unmeasured, Section 6 reports a range of scenario-based estimates rather than one number, and labels every input that is assumed rather than measured. Section 5 describes the remaining measurement work needed to collapse that range to a single reliable figure.

5. What Would Be Needed to Produce a Reliable Estimate

The following measurement steps would need to happen on physical hardware before a battery-life figure could be reported with any confidence:

- Bench-measure current draw in each defined firmware state (sleep, GPS acquiring, LoRa transmitting) on the assembled board, using a current-measurement instrument capable of resolving both the microamp-level sleep current and the higher-current active bursts.
- Log real event counts from bench or field testing — specifically, how many motion-triggered transmissions occur per day for a representative animal — rather than assuming a rate.

- Combine the two: average current (mA) = time-weighted sum of (state current × time in that state) across a full day, incorporating both the fixed 30-minute floor and the measured motion-triggered rate.
- Battery life (hours) = battery capacity (mAh) ÷ average current (mA), using the actual capacity of the cell selected for the final build.
- Report the resulting figure alongside the assumptions it depends on (battery capacity used, measured or modeled current values, assumed event rate) and label it explicitly as an engineering estimate rather than a guaranteed runtime.

6. Battery Life Estimate — 1000 mAh Li-Ion

Battery capacity has now been confirmed as 1000 mAh Li-Ion. This closes one of the three inputs identified in Section 4 as missing; the other two — measured current draw and real-world duty cycle — are still not measured on physical hardware. The figures below therefore use clearly labeled datasheet-typical assumptions in place of measured current, and a small set of duty-cycle scenarios in place of an assumed single event rate, so the result is presented as a range rather than a single number.

6.1 Assumed Parameters (Not Measured)

Parameter	Assumed Value	Basis
Battery capacity	1000 mAh	Stated Li-Ion cell capacity (confirmed for this calculation)
Sleep current	3 μ A	RAK3172 LPM2 (~1.8 μ A) + LIS2DW12 low-power wake mode (~1–2 μ A) — datasheet typical, not measured
GPS acquisition current	25 mA	MAX-M10S acquisition current — datasheet typical, not measured
GPS warm-start time	2 s	Typical warm start with valid ephemeris — assumed
GPS cold-start time	30 s	Typical cold start, no valid ephemeris — assumed
LoRa TX current	120 mA	RAK3172 TX current at 14 dBm — datasheet typical, not measured
LoRa TX time	~70 ms	Calculated airtime for the 18-byte payload at SF9 / CR 4/5

6.2 Calculation Method

For each scenario: GPS energy = GPS current × GPS time × events/day; LoRa energy = TX current × TX time × events/day; sleep energy = sleep current × 24 h. The three are summed to a daily mAh figure, and estimated life = 1000 mAh ÷ daily mAh.

6.3 Scenario Results

Scenario	mAh / day	Est. Days	Est. Months
Warm-start GPS, periodic only (48 events/day)	0.851	1176	~38.7
Warm-start GPS, moderate motion (98 events/day)	1.662	602	~19.8

Warm-start GPS, high motion (348 events/day)	5.717	175	~5.8
Cold-start GPS, periodic only (48 events/day)	10.184	98	~3.2
Cold-start GPS, moderate motion (98 events/day)	20.717	48	~1.6

Reading this table: The roughly 20× spread between the best case (~39 months) and worst case (~1.6 months) is driven almost entirely by whether the GPS performs a warm start or a cold start on each fix — sleep current and LoRa airtime barely move the result by comparison. Once hardware is available, measuring warm-start vs. cold-start GPS behavior across the 30-minute periodic interval is the single highest-value measurement for narrowing this range, ahead of refining the sleep-current or LoRa-current assumptions.

This section remains an estimate, not a specification: all current-draw figures are datasheet-typical values substituted for measurement, and the event-rate scenarios are illustrative rather than derived from observed animal behavior. It should be superseded by the measurement plan in Section 5 as soon as physical hardware is available.

7. Battery Life Estimate — 2000 mAh Li-Ion (Alternative Capacity)

This section repeats the Section 6 calculation for a larger 2000 mAh Li-Ion cell, using the exact same assumed current-draw parameters (Section 6.1) and the same calculation method (Section 6.2). Daily energy consumption per scenario is unchanged — doubling battery capacity does not change how much current the collar draws, only how long a fixed capacity lasts against that draw. Every caveat in Section 6 about assumed, unmeasured values applies equally here.

7.1 Scenario Results (2000 mAh)

Scenario	mAh / day	Est. Days	Est. Months
Warm-start GPS, periodic only (48 events/day)	0.851	2351	~77.3
Warm-start GPS, moderate motion (98 events/day)	1.662	1204	~39.6
Warm-start GPS, high motion (348 events/day)	5.717	350	~11.5
Cold-start GPS, periodic only (48 events/day)	10.184	196	~6.5
Cold-start GPS, moderate motion (98 events/day)	20.717	97	~3.2

Reading this table: Doubling capacity from 1000 mAh to 2000 mAh roughly doubles every runtime figure from Section 6, as expected — the best case moves from ~39 months to ~77 months, and the worst case from ~1.6 months to ~3.2 months. The dominant variable is unchanged: whether GPS warm-starts or cold-starts on each fix still accounts for the majority of the spread, not the choice of battery capacity. A larger cell buys headroom against that uncertainty, but does not resolve it — the measurement plan in Section 5 is still the only way to narrow the range itself.

As with Section 6, this is a scenario-based engineering estimate, not a specification, and should be superseded by measured current draw and observed duty cycle once physical hardware is available.